GENERATIVE AI & AGENTIC AI · LESSON 11 OF 11

GenAI for Data Work: Text-to-SQL, AI Coding Assistants, Data Cleaning and Automated Insights

Apply LLMs to everyday analytics and engineering. Schema-aware text-to-SQL with guardrails and execution-accuracy evaluation, AI coding assistants and agents in notebooks and IDEs, LLM-assisted data cleaning (normalising messy categories, extracting fields from text), generating narrative insights from tables safely, and a workflow that keeps the analyst in control.

● DVA + AI classes 2–3 · GenAI Launchpad

✦ 2026: Claude Code, GitHub Copilot, Gemini in Colab/BigQuery, Copilot in Power BI and Excel

Intermediate ⏱ 120 min

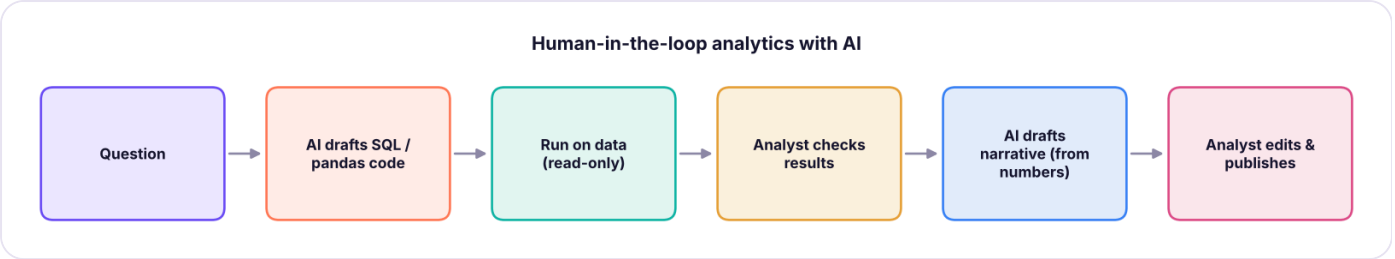
By the end of this lesson you can

- Build a schema-aware text-to-SQL prompt with read-only execution and validation
- Evaluate text-to-SQL with execution accuracy on a gold question set
- Use AI coding assistants effectively and safely for analysis code
- Use LLMs for data cleaning and text-to-table extraction with validation
- Generate narrative insights that are grounded in computed numbers

Datasets used

`superstore_sales.csv` `customer_reviews.csv`

Where GenAI helps analysts



The analyst's workflow with AI assistance at each step, and the human checks that keep it reliable.

Task	Typical tool
Write and explain SQL	Claude/ChatGPT/Gemini, BigQuery and Snowflake copilots, Metabase AI, MinusX
Write Python, fix errors, build charts	Claude Code, GitHub Copilot, Cursor, Gemini in Colab, Jupyter AI
Clean messy text and categories	LLM calls with validation, fuzzy matching
Extract fields from documents (invoices, emails)	LLM structured outputs, document AI services
Summarise dashboards and write reports	Copilot in Power BI, Excel Copilot/Gemini, LLM APIs
Ask questions of data in plain English	Text-to-SQL apps, BI "Q&A" features, data agents

Text-to-SQL

The model needs the **schema**, **business definitions** and **rules**; your code must keep execution **read-only** and **validate** results.

Python

```python
import re, json, duckdb, pandas as pd, numpy as np
sales = pd.read_csv("superstore_sales.csv", parse_dates=["order_date", "ship_date"])
con = duckdb.connect(); con.register("sales", sales)
schema = "\n".join(f"  {c} {t}" for c, t, *_ in con.execute("DESCRIBE sales").fetchall())
PROMPT = f"""You write DuckDB SQL for an Indian office-supplies retailer.

<schema>
TABLE sales (one row per order line)
{schema}
</schema>

<definitions>
- Revenue = SUM(sales); profit margin % = 100 * SUM(profit) / SUM(sales).
- An order is a distinct order_id; AOV = revenue / number of orders.
- Years are calendar years of order_date. Amounts are in INR.
</definitions>

<rules>
- Return ONE SELECT statement only, no comments or explanations.
- Never modify data. Use explicit column names. Round money to 2 decimals.
- If the question cannot be answered from this table, return: SELECT 'cannot answer' AS
message
</rules>

<question>{{question}}</question>"""
print(PROMPT.format(question="Which region had the highest profit margin in 2025?")[:900])
```

OUTPUT

```
You write DuckDB SQL for an Indian office-supplies retailer.

<schema>
TABLE sales (one row per order line)
  row_id BIGINT
  order_id VARCHAR
  order_date TIMESTAMP_NS
  ship_date TIMESTAMP_NS
  ship_mode VARCHAR
  customer_id VARCHAR
  customer_name VARCHAR
  segment VARCHAR
  city VARCHAR
  state VARCHAR
  region VARCHAR
  product_id VARCHAR
  category VARCHAR
  sub_category VARCHAR
  product_name VARCHAR
```

```
  sales DOUBLE
  quantity BIGINT
  discount DOUBLE
  profit DOUBLE
</schema>

<definitions>
- Revenue = SUM(sales); profit margin % = 100 * SUM(profit) / SUM(sales).
- An order is a distinct order_id; AOV = revenue / number of orders.
- Years are calendar years of order_date. Amounts are in INR.
</definitions>

<rules>
- Return ONE SELECT statement only, no comments or explanations.
- Never modify data. Use explicit column names. Round money to 2 decimals.
- If the question cannot be
```

Guarded execution

Python

```
FORBIDDEN =
r"\b(insert|update|delete|drop|alter|create|replace|copy|attach|detach|pragma|install|load|exp
def safe_execute(sql_text, max_rows=200):
    sql_text = sql_text.strip().strip("`").removeprefix("sql").strip().rstrip(";")
    if ";" in sql_text: raise ValueError("multiple statements are not allowed")
    if not re.match(r"^(with|select)\b", sql_text, re.I): raise ValueError("only
SELECT/WITH queries are allowed")
    if re.search(FORBIDDEN, sql_text, re.I): raise ValueError("forbidden keyword")
    return con.execute(f"SELECT * FROM ({sql_text}) LIMIT {max_rows}").df()
print(safe_execute("```sql\nSELECT region, ROUND(100*SUM(profit)/SUM(sales), 2) AS margin
FROM sales WHERE YEAR(order_date)=2025 GROUP BY 1 ORDER BY 2 DESC\n```"))
for bad in ["DROP TABLE sales", "SELECT 1; DELETE FROM sales", "COPY sales TO 'leak.csv'"]:
    try: safe_execute(bad)
    except ValueError as e: print("blocked:", bad, "→", e)
```

OUTPUT

```
  region  margin
0   East   10.93
1  North    9.49
2   West    9.43
3  South    8.50
blocked: DROP TABLE sales → only SELECT/WITH queries are allowed
blocked: SELECT 1; DELETE FROM sales → multiple statements are not allowed
blocked: COPY sales TO 'leak.csv' → only SELECT/WITH queries are allowed
```

Better still: connect with a **read-only database user**, set query timeouts and row limits, and restrict to approved schemas or views.

Evaluate with execution accuracy

Compare the **result** of the generated SQL with the result of a gold query (different SQL can be equally correct).

Python

```
GOLD = {
    "Total revenue in 2024 in crores": "SELECT ROUND(SUM(sales)/1e7, 2) FROM sales WHERE
YEAR(order_date) = 2024",
    "Number of orders in 2025": "SELECT COUNT(DISTINCT order_id) FROM sales WHERE
YEAR(order_date) = 2025",
    "Top 3 states by profit in 2025": "SELECT state FROM sales WHERE YEAR(order_date) =
2025 GROUP BY state ORDER BY SUM(profit) DESC LIMIT 3",
    "Average order value in 2025": "SELECT ROUND(SUM(sales) / COUNT(DISTINCT order_id), 2)
FROM sales WHERE YEAR(order_date) = 2025",
    "Share of lines with discount above 20%": "SELECT ROUND(100.0 * AVG(CASE WHEN discount
> 0.2 THEN 1 ELSE 0 END), 2) FROM sales",
}
GENERATED = {   # what a model returned (one plausible correct, one subtly wrong, one
different-but-correct, ...)
    "Total revenue in 2024 in crores": "SELECT ROUND(SUM(sales) / 10000000, 2) AS
revenue_cr FROM sales WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31'",
    "Number of orders in 2025": "SELECT COUNT(order_id) FROM sales WHERE YEAR(order_date) =
2025",                      # counts lines, not orders
    "Top 3 states by profit in 2025": "SELECT state FROM (SELECT state, SUM(profit) p FROM
sales WHERE YEAR(order_date)=2025 GROUP BY state) ORDER BY p DESC LIMIT 3",
    "Average order value in 2025": "SELECT ROUND(AVG(sales), 2) FROM sales WHERE
YEAR(order_date) = 2025",               # average line value, not order value
    "Share of lines with discount above 20%": "SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE
discount > 0.2) / COUNT(*), 2) FROM sales",
}
def same_result(a, b): return [tuple(r) for r in a.round(2).astype(str).values] ==
[tuple(r) for r in b.round(2).astype(str).values]
rows = []
for q, g in GOLD.items():
    try: ok = same_result(safe_execute(GENERATED[q]), safe_execute(g))
    except Exception as e: ok = False
    rows.append({"question": q, "execution_match": ok})
r = pd.DataFrame(rows); print(r.to_string(index=False)); print(f"execution accuracy:
{r.execution_match.mean():.0%}")
```

OUTPUT

```
                          question  execution_match
Total revenue in 2024 in crores              True
        Number of orders in 2025             False
 Top 3 states by profit in 2025              True
```

```
           Average order value in 2025            False
Share of lines with discount above 20%             True
execution accuracy: 60%
```

The two failures are classic **semantic** errors that run without error: counting order lines instead of distinct orders, and averaging line values instead of order values. That is why business **definitions in the prompt**, a gold test set and analyst review matter more than whether the SQL runs.

Benchmarks such as **Spider** and **BIRD** measure text-to-SQL; real accuracy depends on your schema quality: clear names, documented columns, curated views and a **semantic layer** (dbt metrics, Power BI measures, LookML) dramatically improve results.

AI coding assistants for analysis

Practice	Why
Give context: data shape, column meanings, goal, libraries	Better first drafts
Ask for small steps and run each one	Easier to verify
Ask the assistant to explain unfamiliar code	Learning and review
Make it write tests/assertions (row counts, totals)	Catch silent errors
Never paste secrets or confidential data into tools not approved by your company	Privacy and compliance
Review every line before it touches production	You own the result

Python

```python
# A typical AI-drafted cleaning snippet, with the reconciliation checks you should always add
before_rows, before_sales = len(sales), sales.sales.sum()
clean = sales.drop_duplicates().assign(margin=lambda d: d.profit / d.sales)
clean = clean[clean.sales > 0]
assert len(clean) == before_rows, "row count changed: investigate before continuing"
assert abs(clean.sales.sum() - before_sales) < 1, "revenue no longer reconciles"
print("checks passed:", len(clean), "rows, revenue", f"{clean.sales.sum():,.0f}")
```

OUTPUT

```
checks passed: 6323 rows, revenue 201,413,569
```

Agentic coding tools (Claude Code, Copilot agent mode, Cursor agents) can run whole tasks: explore a repository, write a pipeline, run tests and fix failures. Use them with version control, small

reviewed changes and tests.

LLM-assisted data cleaning

Messy free-text categories are a perfect LLM task: map variants to a controlled list, then **validate**.

Python

```
from difflib import get_close_matches
CANONICAL = ["Bengaluru", "Mumbai", "Delhi", "Hyderabad", "Chennai", "Kolkata", "Pune",
"Ahmedabad"]
raw = ["Bangalore", "bengaluru ", "BLR", "Bombay", "mumbai", "New Delhi", "Dilli", "Hyd",
"Calcutta", "Poona", "Pune-411001", "Gurgaon"]
def rule_based(x):
    m = get_close_matches(x.strip().title().split("-")[0], CANONICAL, n=1, cutoff=0.75);
return m[0] if m else None
llm_mapping = {"Bangalore": "Bengaluru", "bengaluru ": "Bengaluru", "BLR": "Bengaluru",
"Bombay": "Mumbai", "mumbai": "Mumbai", "New Delhi": "Delhi", "Dilli": "Delhi",
              "Hyd": "Hyderabad", "Calcutta": "Kolkata", "Poona": "Pune", "Pune-411001":
"Pune", "Gurgaon": "OTHER"}       # what an LLM prompt returned
out = pd.DataFrame({"raw": raw, "fuzzy_match": [rule_based(x) for x in raw], "llm":
[llm_mapping[x] for x in raw]})
out["llm_valid"] = out.llm.isin(CANONICAL + ["OTHER"])
print(out.to_string(index=False))
```

OUTPUT

```
        raw fuzzy_match       llm  llm_valid
  Bangalore        None Bengaluru       True
 bengaluru    Bengaluru Bengaluru       True
        BLR        None Bengaluru       True
     Bombay        None    Mumbai       True
     mumbai      Mumbai    Mumbai       True
  New Delhi        None     Delhi       True
      Dilli        None     Delhi       True
        Hyd        None Hyderabad       True
   Calcutta        None   Kolkata       True
      Poona        None      Pune       True
Pune-411001        Pune      Pune       True
    Gurgaon        None     OTHER       True
```

Fuzzy matching handles spelling variants but misses old names and abbreviations (Bombay, BLR, Dilli, Calcutta); an LLM with world knowledge handles them, and a validation step guarantees every output is from the allowed list. Gurgaon is correctly flagged as not one of the target cities. For large tables, map the **unique values** once (hundreds of calls, not millions), review the mapping table, and join it back.

Text to table

Python

```
emails = ["Hi, order NM-1042 (laptop) arrived with a cracked screen on 3 Oct. I need a
replacement urgently - Priya",
          "Payment of Rs 2,499 was debited twice for order NM-2210. Please refund one.
Thanks, Arjun"]
extracted = [{"order_id": "NM-1042", "product": "laptop", "issue": "damaged on arrival",
"amount_inr": None, "urgency": "high"},
             {"order_id": "NM-2210", "product": None, "issue": "duplicate payment",
"amount_inr": 2499, "urgency": "medium"}]   # model output via structured outputs
df = pd.DataFrame(extracted)
df["order_id_valid"] = df.order_id.str.fullmatch(r"NM-\d{4}"); df["order_id_in_text"] = [o
in e for o, e in zip(df.order_id, emails)]
print(df.to_string(index=False))
```

OUTPUT

```
order_id product              issue  amount_inr urgency  order_id_valid
order_id_in_text
 NM-1042  laptop damaged on arrival         NaN    high            True
True
 NM-2210    None  duplicate payment      2499.0  medium            True
True
```

Check extracted values against the source text (does the order id actually appear?) and against business rules.

Grounded narrative insights

LLMs write fluent commentary, but numbers must come from your computations, not from the model.

Python

```python
s = sales.assign(year=sales.order_date.dt.year)
k = s.groupby("year").agg(revenue=("sales", "sum"), profit=("profit", "sum"), orders=
("order_id", "nunique"))
k["margin"] = 100 * k.profit / k.revenue
facts = {"revenue_2025_cr": round(k.revenue[2025] / 1e7, 2), "revenue_growth_pct":
round(100 * (k.revenue[2025] / k.revenue[2024] - 1), 1),
         "margin_2025_pct": round(k.margin[2025], 1), "margin_2024_pct":
round(k.margin[2024], 1), "orders_2025": int(k.orders[2025])}
print(json.dumps(facts))
prompt = f"""Write a 3-sentence executive summary for the CFO using ONLY these facts. Do
not add any other numbers.
<facts>{json.dumps(facts)}</facts>"""
draft = ("Revenue reached ₹7.84 crore in 2025, up 19.8% on 2024, from 1,232 orders. "
         "Profit margin improved slightly to 9.6% from 9.4%. Growth was broad-based, so the
priority is protecting margin as volumes rise.")   # an LLM-style draft
nums_in_draft = set(re.findall(r"\d[\d,]*\.?\d*", draft.replace(",", "")))
allowed = {str(v).replace(",", "") for v in facts.values()} | {"2025", "2024", "3"}
print("numbers not in facts:", sorted(nums_in_draft - allowed) or "none")
```

OUTPUT

```
{"revenue_2025_cr": 7.84, "revenue_growth_pct": 19.8, "margin_2025_pct": 9.6,
"margin_2024_pct": 9.4, "orders_2025": 1232}
numbers not in facts: none
```

Pattern: **compute facts in code → pass them as JSON → instruct "use only these facts" → automatically check every number in the draft → human review**. The same pattern drives Power BI Copilot summaries and automated monthly reports.

Summary

KEY TAKEAWAYS

- Text-to-SQL needs the schema, business definitions and rules in the prompt, read-only guarded execution, and evaluation by execution accuracy on gold questions.
- The dangerous errors are semantic (wrong definition) not syntactic; curated views and semantic layers improve accuracy.
- AI coding assistants speed up analysis; give context, work in small steps, add reconciliation checks and review everything.
- LLMs clean messy categories and extract fields from text well when outputs are validated against allowed values and the source.
- Generate narratives from computed facts only, and check every number automatically.

Quick quiz

6 questions · tap an answer to check it

1. Why include business definitions (e.g. "an order is a distinct order_id") in a text-to-SQL prompt?

A Without them the model may write SQL that runs but computes the wrong metric

B To make the prompt longer

C Because SQL requires it

D To speed up the query

Answer: A. Semantic errors are the main risk.

2. What is execution accuracy?

A Whether the generated query returns the same result as a gold query

B Whether the SQL is formatted nicely

C Query speed

D Whether it uses SELECT

Answer: A. Different SQL can be equally correct.

3. Which control best prevents an LLM-generated query from changing data?

A A read-only database user (plus statement checks)

B Asking the model politely

C Higher temperature

D Longer prompts

Answer: A. Least privilege.

4. How should you clean 2 million rows with 300 distinct messy city names using an LLM?

A Map the 300 unique values once, validate the mapping, and join it back

B Call the LLM 2 million times

C Skip validation

D Delete the column

Answer: A. Cheap and reviewable.

5. **An AI-written report says "margin grew 3 points" but your facts show 0.2. What prevents this?**

- **A** Passing computed facts only and automatically checking every number in the draft
- **B** Using a bigger model only
- **C** Writing longer prompts
- **D** Trusting the model

Answer: A. Verify numbers in code.

6. **What improves text-to-SQL accuracy most on a messy warehouse?**

- **A** Clean, documented schemas, curated views and a semantic layer
- **B** More GPU memory
- **C** Removing column names
- **D** Higher temperature

Answer: A. Context quality drives accuracy.

EXERCISE

Practice

(1) Write five more gold questions (including one the table cannot answer) and evaluate a model's SQL with `same_result`. (2) Add a check that rejects queries without a `WHERE` on year when the question mentions a year. (3) Map the `ship_mode` values to an allowed list using fuzzy matching and inspect failures. (4) Generate a monthly KPI facts JSON and write the summary prompt.

Show solution (worked answer)

Python

```
def mentions_year_without_filter(question, sql_text):
    yrs = re.findall(r"20\d\d", question); return bool(yrs) and not all(y in sql_text
for y in yrs)
print(mentions_year_without_filter("Revenue in 2025", "SELECT SUM(sales) FROM sales"))
m = sales.assign(m=sales.order_date.dt.to_period("M")).groupby("m").agg(rev=("sales",
"sum")).tail(2)
facts_m = {"revenue_last_month_lakh": round(m.rev.iloc[-1] / 1e5, 1),
"mom_change_pct": round(100 * (m.rev.iloc[-1] / m.rev.iloc[-2] - 1), 1)}
print(facts_m)
```

INTERVIEW CORNER

Questions you may be asked

Q: How would you build a "chat with your data" tool safely? Curated read-only views with documented columns and metric definitions, schema-aware prompts, guarded execution with timeouts and row limits, show the SQL and results to the user, evaluate with a gold question set, log queries, and route ambiguous questions to analysts.

Q: What are the risks of AI-generated analysis code? Silent logic errors, wrong joins and definitions, security issues, and data leakage to external tools; mitigate with tests, reconciliation checks, code review and approved tools.

Q: How can LLMs help with data cleaning? Normalising categories and entity names, extracting structured fields from text, detecting anomalies in descriptions, and generating cleaning code, always with validation against allowed values and source data.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.